

Cryptography with OpenSSL - Asymmetric cryptography

Laboratory of the course “Computer system security” (02KRQ)
Politecnico di Torino – AY 2014/15
Prof. Antonio Lioy

prepared by:
Diana Berbecaru (diana.berbecaru@polito.it)

v. 2.55 (30/10/2014)

Purpose of the laboratory

The goal of this laboratory is to experiment with the asymmetric cryptography techniques and the main problems encountered when using these techniques. The laboratory makes use of the open-source library named OpenSSL (<http://www.openssl.org/>), which is a cryptographic library released as open-source software with BSD license and available for various platforms, including Linux and Windows.

All the exercises proposed use essentially OpenSSL command line programs. The OpenSSL program allows to use various cryptographic functions by means of the OpenSSL shell, which you can start with the following command:

```
openssl
```

Once you enter the OpenSSL shell, you will have to use the OpenSSL commands (e.g. `genrsa`).

Alternatively, you can use `openssl` followed directly by a command, according to the following syntax:

```
openssl command [ command_opts ] [ command_args ]
```

To perform the exercises on asymmetric cryptography, you will use the following OpenSSL commands:

```
genrsa rsa pkeyutl dgst rand enc
```

These commands are briefly described here below. Additionally, in some exercises, you will need to use also the `openssl` commands `enc` and `dgst`, which have been already presented in the laboratory number 2. For a complete description of the options and for a detailed description of the `openssl` commands, you are advised to consult the corresponding `man` pages.

Additional files and programs required to run some of the exercises proposed below are available at

http://security.polito.it/~lioy/02krq/02krq_1415_lab03_support.zip.

openssl genrsa

To perform simple asymmetric operations with the RSA algorithm, you will have first to generate a pair of RSA keys with the OpenSSL command `genrsa`, whose syntax is:

```
openssl genrsa [-out filename] [numbits]
```

where:

- `-out` saves the generated (public and private) RSA keys in the file named *filename*;

- `numbits` is the length in bits of `N` (i.e. the RSA modulus).

Note: The man pages (`man`) of OpenSSL referring to the RSA private key actually intend both the private key and the public key, since both (RSA) keys are contained in the same data structure. The RSA public key can be separated from the private one by using a dedicated OpenSSL option. In general, however, the keys (public and private) are kept together since we refer to them as “a key pair”.

openssl rsa

To manage and use the RSA keys in cryptographic operations, you can use the OpenSSL `rsa` command, whose syntax is given below:

```
openssl rsa [-in file_input] [-out file_out] [-text] [-pubin] [-pubout] [-noout]
```

where:

- `-in file_input`, specifies the input file (containing an RSA public key or an RSA private key);
- `-out file_out`, saves in the file `file_out` the RSA keys (public or private) after executing the operation requested;
- `-text`, prints the keys in text format. In addition, the keys are also shown encoded in Base64 format unless you use also `-noout`.
- `-pubin`, is a parameter indicating that the key passed in input (via the `-in` option) is a RSA public key. Pay attention, if this parameter is not specified the `rsa` command assumes that the input key is a private one;
- `-pubout`, is the parameter used to generate as output only the RSA public key. Pay attention, if this parameter is not specified, the `rsa` command returns also the RSA private key;
- `-noout`, indicates that the keys in Base64 format do not have to be shown.

openssl pkeyutl

The command `pkeyutl` performs asymmetric encryption/decryption, signature/verification, and key exchange, by using various asymmetric algorithms. Currently, the asymmetric algorithms supported are:

- RSA, to encrypt, decrypt, sign and verify data,
- Diffie-Hellman (DH), for symmetric key exchange,
- Elliptic Curve (EC) to sign and verify with ECDSA and to establish a symmetric key with ECDH.

```
openssl pkeyutl [-encrypt] [-decrypt] [-in file_input] [-out file_output] [-pubin]
                [-inkey file_key] [-sign] [-verify] [-verifyrecover]
```

where:

- `-in file_input`, specifies the input file (containing the message to encrypt, decrypt, sign or verify);
- `-out file_out`, saves the output of `pkeyutl` in `file_out`;
- `-inkey file_key`, specifies that the file `file_key` contains the public key or the private key;
- `-encrypt/-decrypt`, encrypts with the public key or decrypts with the private key (the content of) the input file whose name is passed with the parameter `-in`;

- `-pubin`, parameter indicating that the key passed in input (with the option `-inkey`) is a public key. Pay attention, if this parameter is not specified `pkeyutl` assumes that a private key is passed as input;
- `-sign`, generates the signature applied on the input file (passed with `-in`) by using the key passed in with the option `-inkey`. A private key is required in this case. More precisely, if an RSA key is used, the file passed in input is encrypted with the private key;
- `-verifyrecover`, verifies the signature passed in as the input file (that is the file passed in with the option `-in`) by using the key passed in with the option `-inkey` and shows the decrypted data. Pay attention that an RSA public key is required (the command does not function with DSA, DH or ECC). This option is available only if an RSA key is used, in practice, the input file is decrypted with the public key;
- `-verify`, computes the signature on the input file (passed in with `-in`) by using the public key passed in with the option `-inkey` (a public key is required) and compares this signature with another signature passed in with the option `-sigfile`. Returns a boolean value (*Signature Verified Successfully/Signature Verification Failure*).

Note: If in the most OpenSSL commands the order of parameters is not important, in `pkeyutl` the order in which the parameters are passed is important. We suggest you to follow the order of parameters presented above, otherwise the execution of the `pkeyutl` command will fail.

openssl dgst

To compute and verify a digital signature, another simpler command to use is the OpenSSL command `dgst`:

```
openssl dgst [-md5|-md4|-md2|-sha1|-sha|-mdc2|-ripemd160|-dss1]
             [-out filename] [-sign private_key] [-verify public_key]
             [-signature signature_file] [file_to_sign]
```

where:

- `-sign private_key`, calculates the signature of the file `file_to_sign`, by using the hash algorithm specified as option and the private key contained in the file `private_key`;
- `-verify public_key`, verifies the signature applied on the file `file_to_sign`, by using the digest algorithm (specified as option) and the public key contained in the file `public_key`. For this operation, it is mandatory to specify which is the signature file, by using the option `-signature`;
- `-signature signature_file`, indicates which is the file containing the signature to be verified, in case the option `-verify` is used.

openssl rand

This command generates `nbytes` pseudo-random bytes and saves them in the file `file_name`:

```
openssl rand -out file_name nbytes
```

For a detailed description of the above commands, you can use the relative `man` pages.

SSH server and SCP utility

In the exercises, you will be asked to exchange messages between two computers, by using the `scp` utility. Consequently, you have to start the `ssh` server:

```
/etc/init.d/ssh start
```

1 Asymmetric cryptography operations

1.1 Key generation

Starting from the OpenSSL command `genrsa`, discover and write down the OpenSSL command used to generate a 2048 bit RSA key pair, and save it in the file `rsa.key`:

Once you have generated the RSA key pair, check out the content of the file `rsa.key` with the following command:

```
openssl rsa -in rsa.key -text
```

What is the purpose of the parameters `modulus`, `publicExponent`, `privateExponent`, and `prime*`?

Which of the parameters can be made public (are part of the public key) and which ones instead must be kept secret (are part of the RSA private key)?

Compare the parameters with the ones generated by your colleague: do you notice any common parameters? If yes, which is their purpose?

Suppose you want to distribute your new RSA public key to your colleagues: write down the OpenSSL commands used to extract the RSA public key from the file `rsa.key` to the file `rsa.pubkey`, and to view its content:

1.2 Encryption and decryption operations with RSA asymmetric algorithm

In this exercise, you will encrypt/decrypt a block of data by using the RSA algorithm and the RSA key that you have generated in the previous exercise. Create a text message, like for example “This is a confidential message” (without the “ and ” characters), and save it in a file named `plain`.

How can you use your RSA key pair to ensure confidentiality of the file `plain`? Which (RSA) key do you have to use?

Write down the OpenSSL command to encrypt the file `plain`, and save the result in the file `encRSA` (suggestion: use the command `pkeyutl`).

Which operation performs the following OpenSSL command and which key is used in this command?

```
openssl rsautl -encrypt -in plain -inkey rsa.key -out encRSA
```

Write down the command used to decrypt the message encrypted above:

Extract the files `rsaplain*` from the support archive and encrypt them as above. Do you face any problem? Why (see the note below)?

Note: RSA allows in theory to encrypt any message, which interpreted as a binary value is smaller than the value of the modulus (that is a string of 2048 bits for a 2048-bit RSA key). Nevertheless, the PKCS#1 format imposes additional limitations that are due to the encapsulation of the message to be encrypted in a PKCS#1 envelope, and in particular due to the padding required. In practice, if you have an N -bytes RSA key, you can perform successfully encryption/decryption operations with OpenSSL only if the (plaintext) data is long at most $N - 11$ bytes.

1.3 Signing and verifying operations with RSA asymmetric algorithm

The command `pkeyutl` allows not only to encrypt/decrypt blocks of data, but also to sign/verify them. Which is the difference in terms of the RSA operations to be performed?

Write down the OpenSSL `pkeyutl` command used to sign the file `plain`, and save the signature in the file `sig`. Next, find out and write down the OpenSSL command used to verify the signature contained in the file `sig`, by using again the `pkeyutl` command.

Which keys did you use for each of the above operations?

2 Application of asymmetric cryptography

2.1 Session keys

Suppose Alessandro and Beatrice want to exchange data messages encrypted with AES symmetric algorithm, but they do not have a shared secret key. Form two groups: do you manage to design a solution based on the OpenSSL commands that you have used so far (for symmetric and asymmetric operations), so that Alessandro manages to send the file `rsaplain2` encrypted with AES to Beatrice even though Alessandro and Beatrice do not have a common shared secret key? (pay attention: Beatrice must be able to decrypt correctly the ciphertext received from Alessandro)

Suppose Alessandro proceeds in the following manner:

- executes:

```
echo key > aeskey
openssl enc -e -aes-128-cbc -in rsaplain2 -out msg.enc -kfile aeskey -iv 0
openssl pkeyutl -encrypt -in aeskey -inkey rsa.key -out aeskey.enc
```
- sends `msg.enc` and `aeskey.enc` to Beatrice
- sends `rsa.key` to Beatrice

Which serious errors did (inexperienced) Alessandro? (there are at least 2!)

Using the OpenSSL command `enc` and `pkeyutl`, write down the complete and correct sequence of OpenSSL operations that Alessandro and Beatrice must execute:

What kind of security attack could do Caterina to intercept the data messages exchanged between Alessandro and Beatrice? How can you avoid the attack Caterina could do to intercept the data messages exchanged between Alessandro and Beatrice? (Hint: think about the security of the first step in the workflow above.)

Which other asymmetric algorithm could use Alessandro and Beatrice instead of RSA algorithm in order to perform key exchange?

What are the operations that have to be performed in this case in order to exchange data messages securely ? Considering that you identified the alternative asymmetric algorithm (instead of RSA) for key exchange, what kind of security attack could (still) do Caterina to intercept the data messages exchanged between Alessandro and Beatrice? How can you avoid the attack Caterina could do to intercept the data messages exchanged between Alessandro and Beatrice?

2.2 Digital signature

Suppose Alessandro wants now to protect the integrity of the messages sent to Beatrice (instead of protecting the confidentiality of the messages). Form two groups: design a solution that allows Alessandro to sign the file `rsaplain2`, and to Beatrice to verify the signature applied on the file `rsaplain2`.

Using the commands `dgst` and `pkeyutl`, write down the complete and correct sequence of OpenSSL commands that Alessandro and Beatrice must execute to implement your solution:

What happens if Caterina modifies one of the blocks of data sent by Alessandro to Beatrice? In practice, if Caterina modifies the original message (but not the signature), which will be the result of signature verification?

If Caterina modifies the signature (but not the data), which will be the result of signature verification?

What happens if Caterina replaces the public key of Beatrice with her own public key? Can she replace the original data and the signature? How will be the result of the signature verification in this case ?

Note that, besides the command `pkeyutl`, you could use directly the command `dgst` to perform the signing and verifying operations. For example, you can construct the digital signature of the file `msg` with the OpenSSL command:

```
openssl dgst -sha1 -sign RSAkey -out msg.sig msg:
```

Write down the `dgst` OpenSSL command used to verify the signature:

What differences do you note when you use the `pkeyutl` command instead of the `dgst` command?

2.3 Authentication

In this exercise, you'll use the RSA functions (and the `pkeyutl` command) to implement a challenge-response protocol.

Form two groups (Alessandro and Beatrice) and proceed in the following manner:

- Alessandro sends to Beatrice his own public key.
- Beatrice generates a random string in the file `random` and encrypts it with the public key of Alessandro: the result obtained is the file `challenge`. Write down the OpenSSL commands required to perform this step, by using using the commands `rand` and `pkeyutl`:
- Beatrice sends the file `challenge` to Alessandro.
- Alessandro decrypts the challenge contained in the file `challenge` and sends the response to Beatrice.
By using the `pkeyutl` command, write down the commands required to complete this step (save the response to the challenge in the file `response`):
- Beatrice verifies that the response to the challenge is correct (you can use the command `diff -b`).

Assuming that the keys are authentic, which is the weakness of the challenge-response protocol executed above? Which are the possible solutions? (at least two solutions!)

2.4 Digital certificates

The signatures constructed and experimented in the exercises proposed in [2.1](#), [2.2](#), [2.3](#) are subject to a common vulnerability: if a malicious attacker is able to intercept and modify the exchange of the public keys, he can compromise the security of the entire system.

The objective is always the same: Alessandro and Beatrice want to exchange data securely. Suppose that Alessandro and Beatrice have a common friend, Carlo. Carlo knows the public keys of Alessandro and Beatrice, and they know the public key of Carlo.

How can Carlo help his friends (Alessandro and Beatrice) to exchange messages securely? Hint: Carlo might act as an “authority” deriving from the mutual knowledge of the public keys.

Suppose Carlo proceeds as follows:

- modifies the file containing the public key of Alessandro, `ale.pubkey`, by adding the text:

```
echo "This is the public key of Alessandro, Carlo grants for it" >> ale.pubkey
```
- Carlo signs (with the command `dgst`) the modified file (`ale.pubkey`) containing the public key of Alessandro and the text added by Carlo.
- Carlo sends to Alessandro the file modified and the generated signature.

Subsequently, Carlo does the same with the file containing the public key of Beatrice.

Do you manage to adapt the protocols described in the exercises described in Sections [2.1](#), [2.2](#) e [2.3](#), so that to exploit the files created by Carlo? What additional operation must perform Alessandro and Beatrice in case Carlo is acting as trusted party for the public keys (of Alessandro and Beatrice) ?

Form a group of three hosts (Alessandro, Beatrice, Carlo), choose one of the modified protocols and execute it.

2.5 Performance

Use the command `openssl speed` to perform performance comparisons among:

- RSA keys of 512, 1024, 2048 and 4096 bits in length.
How does the performance decreases in terms of the key length?
What is the difference (in terms of performance) between operations requiring the use of the public key and the ones requiring the use of the private key?
- digital signature algorithms: 1024 bit RSA, 1024 bit DSA, 160 bit ECDSA (`ecdsap160`).
What differences do you note? Are the ECC operations more efficient than RSA/DSA ones? Is it more efficient to sign or to verify with DSA/ECDSA? Is it more efficient to sign or to verify with RSA?
- high security algorithms: 2048 bit RSA, 2048 bit DSA, 224 bit ECDSA (`ecdsap224`).
What differences do you note between the results obtained at this step and the ones obtained at the previous step? How does the performance decreases for RSA/DSA and for ECDSA with the increase of the key length (and thus of the security strength)?
- symmetric, asymmetric cryptography and digest: 2048 bit RSA, 2048 bit DSA, 128 bit AES and SHA-256. (Note: as above, 128 bit AES guarantees a security level equivalent to 3072-bit RSA/DSA.)
How can you compare the results obtained?
How much faster are symmetric cryptography operations (`aes-128-cbc`, `sha256`) when compared to asymmetric operations (`rsa2048`, `dsa2048`)?

3 Putting all together (optional)

Based on the knowledges you acquired on symmetric and asymmetric cryptography, think at a solution for the following projects.

3.1 Message Protection

Suppose that Alessandro has the RSA public key of Beatrice, and wants to use it to send her via e-mail a file for which he wants to protect the confidentiality and integrity. Identify the various blocks of data that Alessandro must create, define a format to encapsulate the various blocks of data in a single mail and create the envelope.

Hints:

- for the secure data format used, you can get inspiration from PEM, used by OpenSSL to save RSA keys. PEM uses textual separators and represents binary data as text, by using the `base64` encoding. The `base64` encoding/decoding can be done by means of the OpenSSL command `enc`.
- which blocks of data must be encrypted?
- which blocks of data must be signed?
- which are the differences in terms of security properties among (1) encrypting the message and then signing the encrypted message, or (2) signing the message in clear and then encrypting the message together with its signature?

3.2 Channel protection

Suppose that Alessandro has the RSA public key of Beatrice: design a protocol that allows Alessandro and Beatrice to exchange messages over an insecure channel, so that to guarantee the confidentiality and integrity of the data exchanged.

Alessandro e Beatrice needs to ensure the data exchange is confidential:

- how much data Alessandro and Beatrice need to exchange?
- which cryptographic algorithms need to be used to ensure confidentiality of the data?

Considering that you have individuated a solution to ensure data confidentiality:

- which problem remains open (and needs to be solved) at this point?
- what can do Alessandro to solve this problem, considering the information he knows about Beatrice?

Write down the solution is most appropriate in this case (in your opinion):

Alessandro and Beatrice need to ensure also the integrity of the data exchanged:

- what methods do they have at disposal?
- do you think it is possible to ensure also the message authentication? If yes, with which method?

Specify a method that you think is appropriate to ensure integrity and data authentication:

At this point, you can put together the above responses and think at a protocol for channel security.